

Learning what makes a scaffolded epitope bind: DP4 assay features, models, and how they compare to the composite

Episcaf v2: DP4 IM0276 readout

2026-09-22

Abstract

The DP4 assay gave us, for the first time, a real binding readout on thousands of designs whose metrics we already had. This document collects what we learned from it: which features separate binders from non-binders, which model family predicts binding best, and how a fitted model compares to the hand-set composite score and Lawson’s four-filter. The composite score was already the right shape (a hand-weighted logistic model), refitting its weights on the assay adds almost nothing, and the gain comes from adding features of the epitope’s own native structure. The most predictive of those features is rigidity in the bound state. Because that signal disappears once the antibody is removed, it raises an apo-versus-holo question the static structures cannot settle.

1 The data and what we predict

The assay is a PepSeq enrichment readout (antibody IM0276 against the DP4 library). We joined it to the design library on the row-id ($\text{DP4_}\langle n \rangle_ \langle m \rangle \rightarrow \text{DP4_}\langle n \rangle$, 100% merge). A design counts as a *binder* if it appears in the assay hit list for its epitope’s antibody.

Across the scaffolded arms, 2,720 designs carry both a full set of design metrics and an assay call. Of those, 217 bind, a base rate of 8.0%. They come from 56 epitopes, each with its own antibody. That epitope structure matters for how we score every model: we always split by epitope in cross-validation, so a model is tested on antibodies it never trained on. Scoring within a single epitope would leak, because the antibody identity, not the design, carries most of the enrichment.

2 The composite was already a logistic model

The composite score takes each design metric, passes it through a saturating sigmoid, multiplies by a hand-set weight, and sums. Designs that clear all four Lawson filters get a fixed promotion on top. That is the functional form of a logistic regression with a per-input nonlinearity, and the code says as much (`episcaf_analysis/presets.py`: “a single-layer perceptron with a per-input nonlinearity and hand-set weights...to learn the weights instead, fit a logistic regression”).

So fitting a logistic regression is the same object with two changes: the weights are fit to the assay instead of set from the DP3 prior, and it can take extra inputs. The ablation below shows which change helped.

3 Features

We group the inputs into three blocks (Table 1). The design metrics are what we already compute per design. The secondary-structure counts come from John’s per-epitope helix, strand, and loop tallies. The native-context and dynamics block is new: we read it off each epitope’s own antibody:antigen crystal structure from the RCSB.

Table 1: The three feature blocks. The native-context block is read from each epitope’s deposited complex with `biotite`. To locate the epitope inside each complex without guessing chain roles, we take the antigen chain whose antibody-interface secondary structure best matches John’s independent helix/strand/loop counts; that match also validates the extraction (Pearson $r = 0.93$ against John’s helix fraction, MAE 0.10, $n = 56$).

Block	Features	Source
Design metrics (9)	cylinder_clashes, epitope_rmsd, overall_rmsd, epitope_pae, scaffold_pae, mean_pae, ptm, af3_clashes, island_index	pipeline (per design)
Secondary structure (3)	helix, strand, n_islands	John’s counts (per epitope)
Native context & dynamics	epi_bfac_z (bound-state B-factor rigidity), epi_gnm_apo / _holo / _dfit (network-model flexibility, antigen alone vs. in complex), epi_dsasa (surface buried on binding), epi_size, epi_rel_sasa, epi_rg, composition	RCSB complex (per epitope, <code>biotite</code>)

4 Which features and which model help

Table 2 and Figure 1 answer both questions at once: we add feature blocks left to right, and we swap the model family down the rows.

The features drive the result, and the model family matters much less. Fitting a logistic regression on the same design metrics the composite already uses gives ROC-AUC 0.741, barely above the composite’s 0.733. Adding the secondary-structure counts takes it to 0.783, and adding native context and dynamics takes it to 0.810.

The model family barely matters as long as it is simple. Logistic regression (0.810) and random forest (0.811) tie at the top with the full feature set, and gradient boosting is close behind (0.796). The multilayer perceptron is worst at every feature set and actually gets worse as we add features, which is what a small, noisy dataset does to a model with too many parameters.

5 The signal is additive: feature interactions do not help

The ablation adds feature blocks in one fixed order, so the block added last looks smallest only because the others are already in. The order-free version is each block’s Shapley value: its average marginal ROC-AUC gain over all orderings of the three blocks, which with three blocks is only eight subsets to score (Table 3). Secondary structure is the largest single block, ahead of the nine

Table 2: Feature ablation crossed with model family. The columns are cumulative: each is the previous plus one block. *design metrics* (9) are the per-design pipeline outputs (cylinder_clashes, epitope_rmsd, overall_rmsd, epitope_pae, scaffold_pae, mean_pae, ptm, af3_clashes, island_index); + *secondary structure* adds the epitope’s helix, strand, and island counts; + *dynamics* adds the native-context features epi_bfac_z (bound-state rigidity), epi_gnm_dfit (change in flexibility on binding), and epi_dsasa (surface buried on binding). ROC-AUC under repeated stratified grouped cross-validation (split by epitope), mean $\pm$ sd over shuffles; composite baseline 0.733. Adding each block helps every model except the MLP.

Model	design metrics	+ secondary structure	+ dynamics
Logistic regression	0.741 $\pm$ 0.022	0.783 $\pm$ 0.008	0.810 $\pm$ 0.013
Gradient boosting	0.729 $\pm$ 0.033	0.781 $\pm$ 0.019	0.796 $\pm$ 0.019
Random forest	0.720 $\pm$ 0.032	0.791 $\pm$ 0.014	0.811 $\pm$ 0.013
MLP (32, 16)	0.672 $\pm$ 0.024	0.646 $\pm$ 0.014	0.654 $\pm$ 0.013

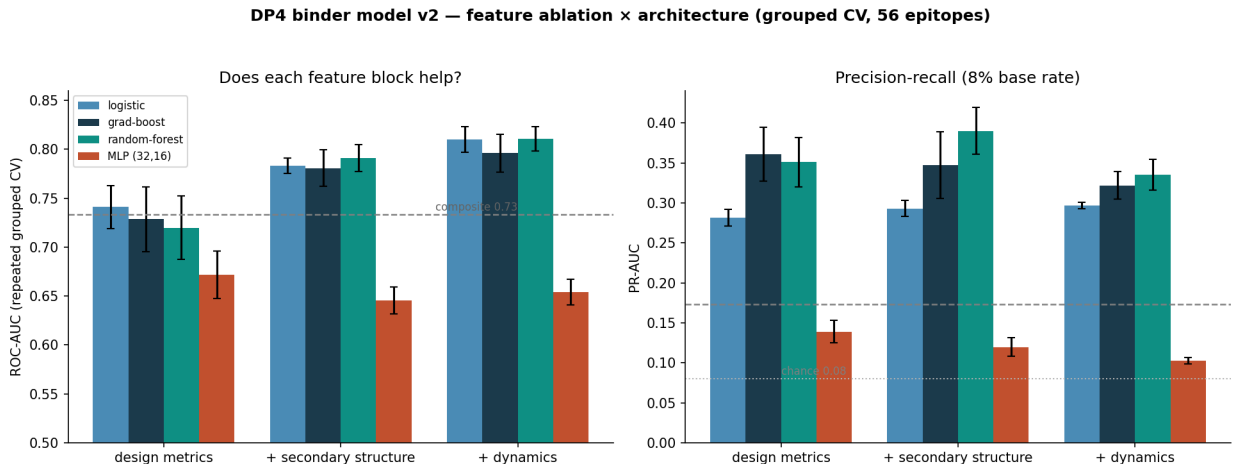


Figure 1: Feature ablation by model family. Left: ROC-AUC as we add feature blocks (x-axis), one bar group per model, error bars over repeated grouped-CV shuffles; the dashed line is the composite baseline. Right: the same models scored by PR-AUC, which is the fairer view at an 8% base rate (the dotted line is chance, 0.08). The pattern is the same in both panels: each feature block lifts the tree and linear models, the MLP trails, and no model beats a simple one once it has the full feature set.

design metrics, and on its own it reaches 0.767 of the full model’s 0.796. The three values plus the 0.5 baseline sum to the full-model AUC, so the attribution is exact.

Shapley still *adds* the blocks; it does not ask whether *combining* features beats adding them. We test that directly. The gradient booster can be constrained to forbid all cross-feature interactions, which leaves a purely additive model that is still nonlinear in each feature. Sweeping tree depth with interactions forbidden versus allowed (Table 4), the additive model is as good or better at every depth, and the best model of all is depth-1 stumps, where each tree uses a single feature and no interaction is possible. The lift over plain logistic regression (0.809 to 0.846) comes entirely from letting each feature take a nonlinear shape, not from interactions.

An Explainable Boosting Machine, a glassbox additive model that adds only the pairwise interactions that earn their place, confirms this from the other direction: additive-only scores 0.834, and adding up to ten pairwise interactions drops it to 0.829. Those interaction terms carry 11% of the model’s importance measured in-sample, yet they lower the held-out AUC. That is a useful reminder in itself: interaction importance read off the training data is usually overfitting, and only

Table 3: Shapley value of each feature block for gradient boosting: the average marginal ROC-AUC gain over all orderings of the three blocks (repeated grouped CV). On its own, secondary structure scores 0.767, design metrics 0.729, dynamics 0.671; the full model is 0.796. The values plus the 0.5 baseline sum to the full-model AUC, so no block’s contribution depends on the order we happen to add it.

Feature block	Shapley value (avg. marginal ROC-AUC)
Secondary structure (helix, strand, island count)	+0.128
Design metrics (9)	+0.109
Dynamics (rigidity, Δflex , ΔSASA)	+0.059
sum + 0.5	0.796 (= full-model AUC)

Table 4: Do feature interactions help? ROC-AUC (repeated grouped CV) as tree depth grows, with cross-feature interactions forbidden (a purely additive model) versus allowed, at matched per-feature flexibility. Logistic regression (linear and additive) scores 0.809. Forbidding interactions never costs and usually helps; the best model is depth-1 stumps, additive and nonlinear.

max tree depth	interactions forbidden (additive)	interactions allowed
1	0.846	0.846
2	0.838	0.822
3	0.821	0.805
4	0.794	0.793
6	0.786	0.783
8	0.769	0.783

grouped-CV generalization counts. The EBM’s largest main effects are secondary structure (strand and helix), the same conclusion the Shapley analysis reached.

Figure 2 shows the per-feature response the additive model learns. Read each curve on its own and add them up; the model’s output is the sum of these fifteen shapes. They are gentle bends, not steps or spikes, which is why a mildly nonlinear additive model captures the signal while a deeper, interacting one only overfits.

Together these say the binding signal is an additive, mildly nonlinear scorecard dominated by the epitope’s secondary structure, with no feature interactions that survive out-of-fold. The useful model is interpretable rather than a black box, and this vindicates the additive form of the composite. One caveat: at 56 epitopes a subtle real interaction could sit below detection, so this is “no interaction we can measure,” not “none exists.”

6 Does any model overfit, and which generalizes best?

A single held-out split of twelve epitopes is too noisy to rank models, so we repeat it: 20 random diverse splits, each starting farthest-point sampling from a different seed, each holding out 6 binder and 6 non-binder epitopes (median 53 test binders per split), with the same feature set as the modeling section. Table 5 and Figure 3 report train and held-out AUC as mean $\pm$ sd over the splits.

Overfitting shows up as the gap between train and held-out AUC. The oversized MLP has the largest gap (train 0.995, held-out 0.776, gap 0.219), and its held-out AUC is no better than a plain logistic regression despite far more parameters. The composite, which is never fit, has no gap at

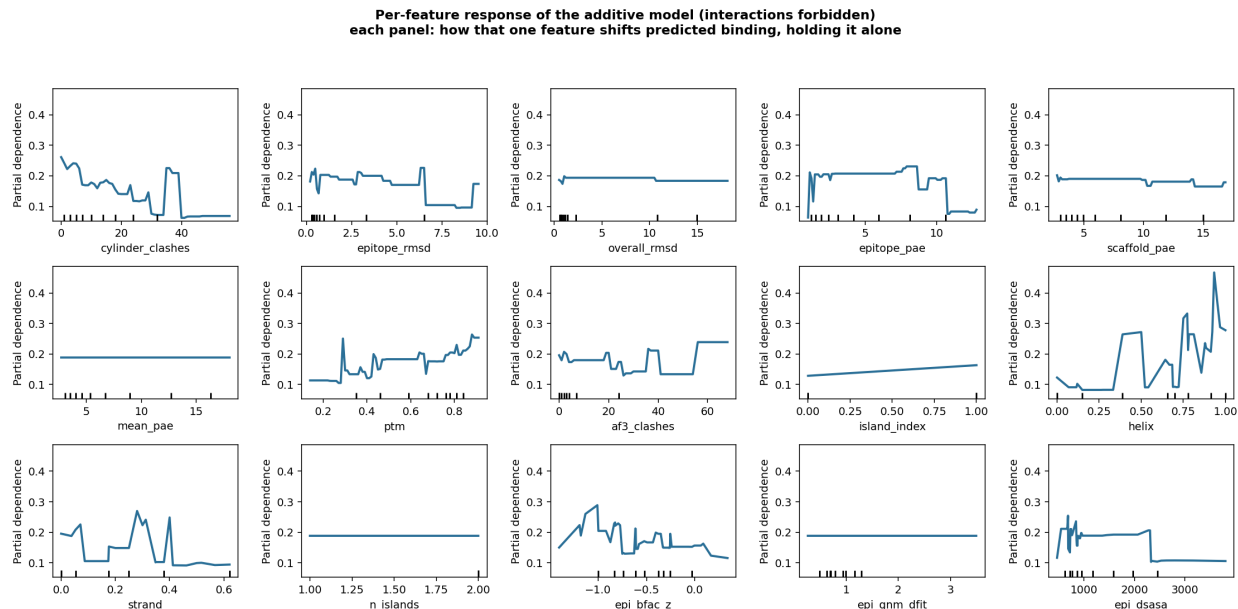


Figure 2: Per-feature response of the additive model. Each panel is one feature’s learned contribution to predicted binding, with interactions forbidden, so the model’s score is the sum of these fifteen curves. The bends are why the additive-nonlinear model beats linear logistic; their mildness is why adding interactions does not help.

all, with held-out AUC slightly above train. On unseen diverse epitopes the fitted models cluster between 0.76 and 0.80. Gradient boosting has the best mean and the lowest variance (0.796 ± 0.030), but the spreads overlap, so the differences among gradient boosting, random forest, logistic, and the composite are within about one standard deviation. The small MLP is the one clear laggard (0.652).

The repetition also settles the question a single split had raised. On one split logistic looked much worse than the trees, but that was a small test set and a mismatched feature set. Repeated with the same features, logistic recovers to 0.757 ± 0.029 , within a standard deviation of the trees. That is consistent with the cross-validation in Table 2: simple models and trees are close, the MLPs do not help, and the unfit composite is competitive. The two tables differ only in that Table 2 tests on representative random folds while this one tests on deliberately diverse, harder epitopes, which is why the held-out numbers here sit a little lower.

Table 5: Repeated held-out head-to-head: mean $\pm$ sd over 20 random diverse splits (6 binder + 6 non-binder epitopes each, same features as Table 2). A large train-minus-held-out gap is overfitting. The oversized MLP has the largest gap and no held-out gain over logistic; the unfit composite has no gap. On held-out epitopes the fitted models cluster within about one standard deviation, gradient boosting best in mean and variance.

Model	train AUC	held-out AUC	gap
Composite (no fit)	0.724 ± 0.014	0.773 ± 0.051	-0.049
Logistic regression	0.905 ± 0.008	0.757 ± 0.029	$+0.148$
Gradient boosting	0.996 ± 0.001	0.796 ± 0.030	$+0.201$
Random forest	0.981 ± 0.002	0.766 ± 0.046	$+0.215$
MLP (right-sized)	0.695 ± 0.024	0.652 ± 0.070	$+0.043$
MLP (oversized)	0.995 ± 0.002	0.776 ± 0.048	$+0.219$

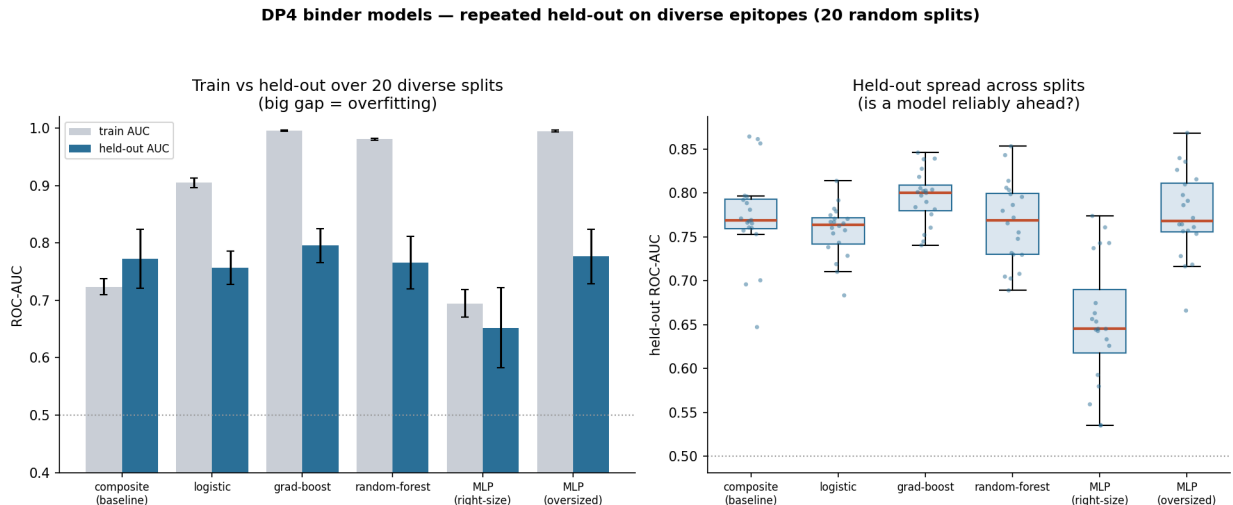


Figure 3: Repeated held-out over 20 diverse splits. Left: train (grey) and held-out (blue) ROC-AUC, mean $\pm$ sd across splits; the taller the drop from grey to blue, the more the model overfits. The oversized MLP has the largest drop and no held-out advantage over logistic, while the unfit composite does not drop at all. Right: the spread of held-out AUC across the 20 splits for each model (box = quartiles, red line = median, dots = individual splits). The fitted models overlap heavily between 0.75 and 0.80, so no single one is reliably ahead; the small MLP is the clear laggard.

7 Binary label beats a pseudo-affinity target

We also asked whether training on a graded target does better than a yes/no label. From the assay dilution series we built a pseudo-affinity per design (a titration score standing in for ΔG , since $\Delta G = RT \ln K_d$ is a monotone transform of K_d). On the 2,412 designs of the 25 antibodies with a usable dilution series, a classifier trained on the binary hit label ranks binders slightly better than a regressor trained on the pseudo-affinity (ROC-AUC 0.665 vs 0.633, Figure 4). The regressor barely recovers the graded affinity at all (Spearman between prediction and pseudo-affinity is -0.10). Graded affinity, in this readout, is close to unpredictable from our features; the binary in-or-out question is the one the data can answer.

8 The strongest new feature is rigidity in the bound state

Comparing the 21 binding epitopes to the 35 non-binding ones on the native-context features shows the largest separation of any feature we tested. Binders sit at lower bound-state B-factor (median z-score -0.79 vs -0.33 , Mann-Whitney $p < 0.001$): the epitopes that scaffold well are the rigid ones in the crystal. The network-model flexibility of the epitope inside the complex tells the same story ($p = 0.001$). Binders also bury less surface on binding (median ΔSASA 891 vs 1255 $\text{\AA}^2$, $p = 0.010$), consistent with a pre-formed epitope that does not have to reshape a large interface.

The caveat is what does *not* separate. The apo network-model flexibility, computed with the antibody removed, does not distinguish binders from non-binders at all (median 0.20 vs 0.27, $p = 0.76$). Every rigidity signal that works is measured in the bound state, where the antibody is already holding the epitope in place. So we cannot yet tell whether binders are intrinsically pre-organized before the antibody arrives, or whether they only look rigid because they are clamped.

DP4: binary label vs pseudo-affinity (dG) as the training target

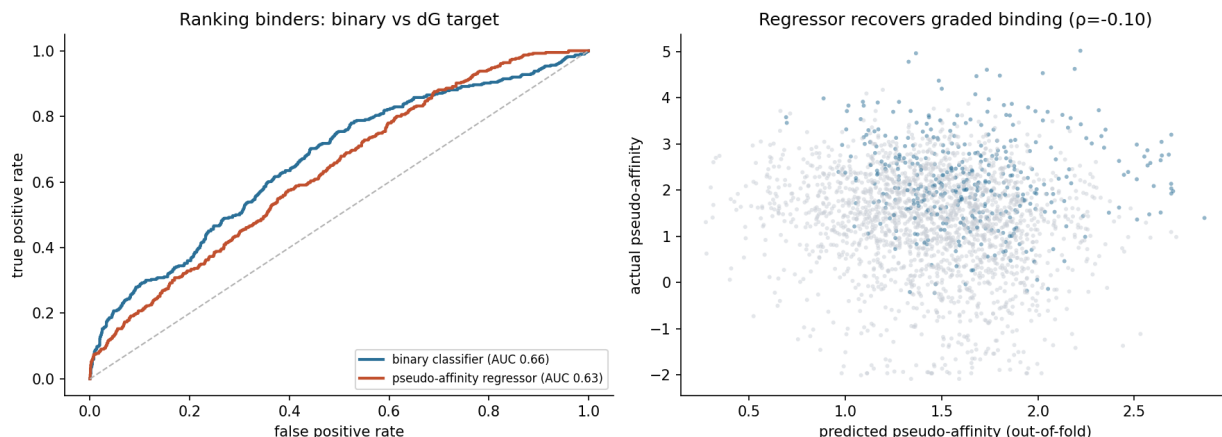


Figure 4: Binary label versus pseudo-affinity as the training target. Left: ROC for ranking binders, binary classifier versus pseudo-affinity regressor, same designs and same grouped CV; the binary target wins. Right: the regressor’s predicted pseudo-affinity against the measured value, with binders highlighted; the cloud is nearly flat, so the graded signal is not there to be learned. We train on the binary label from here on.

This is the apo-versus-holo question the v3 simulations are set up to answer, and it is a concrete reason to run apo dynamics on these epitopes, since the static structures cannot separate the two cases.

9 The right way to scaffold depends on the epitope

Brandon’s standing hypothesis was that there is no single rule for good scaffolding, and that the right metric to optimize depends on the epitope’s native context. Figure 5 tests this directly. For each design metric we take its within-antibody correlation with binding (the honest design-level signal, with the antibody effect removed), computed separately inside each context.

The composite predicts binding for helical epitopes (within-antibody correlation +0.23) and not at all for loopy ones (+0.03). Accessibility (cylinder_clashes) predicts strongly for rigid and helical epitopes (−0.22) and weakly for flexible ones (−0.08). The AF3 clash term even flips sign between helical (−0.19) and loopy (+0.04) epitopes. The design metrics work where the epitope is structured and rigid, and they lose their grip where it is loopy and flexible. This supports the hypothesis: the scaffolding rule is context-dependent, so a context-aware model fits the data better than one global rule.

10 How the models compare to the composite and the four-filter

As a tool for deciding which designs to make, how do the fitted models compare to the composite and to Lawson’s four-filter? We rank designs by each score and screen from the top down. The composite and the fitted models each give a continuous score, so each traces a curve; the four-filter is a yes/no rule, so it gives a single operating point. We show both model families, logistic

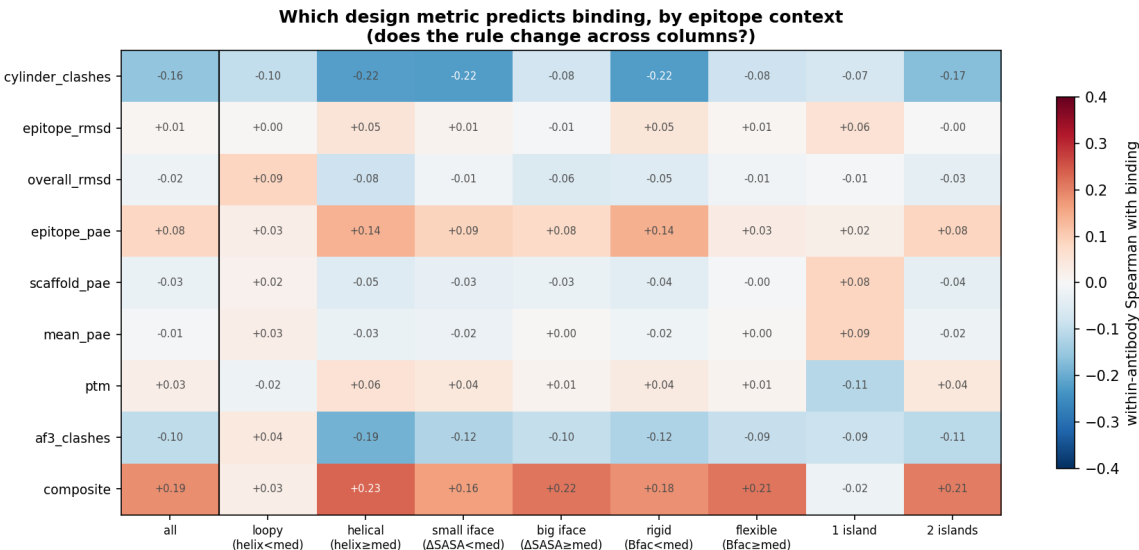


Figure 5: Which design metric predicts binding, by epitope context. Each cell is the within-antibody correlation between a design metric (rows) and binding, computed inside one context stratum (columns); blue means lower metric goes with more binding, the expected direction for clashes and RMSD. Reading across a row shows whether a metric’s usefulness changes with context. The clash and composite rows change sharply between helical/rigid columns and loopy/flexible ones, which is what a context-dependent scaffolding rule looks like.

regression and gradient boosting, each with design metrics only and with the full feature set, so the plots show the model choice and what the features add at the same time.

Figure 6 reads as “screen the top X% of designs, catch Y% of the binders.” At the top 20%, the composite catches 44% of the binders, logistic with all features 60%, gradient boosting with all features 67%, and random picking 20%. The features help gradient boosting the most: its top-20% capture rises from 58% with design metrics only to 67% with the full set. For logistic the features show up more in overall AUC than at the very top of the ranking. The four-filter passes 13% of designs and catches 39% of the binders, an elbow that sits on the composite curve, because the composite’s global-pass promotion is a soft version of the same four filters.

Figure 7 is the same comparison as ROC curves. Adding the native-context features lifts both families by about 0.06 AUC (logistic 0.76 to 0.82, gradient boosting 0.76 to 0.83), and with the full set the two are within 0.01 of each other. Every fitted curve passes above and to the left of both the composite curve (AUC 0.73) and the four-filter point (false positive rate 0.11, true positive rate 0.39), so the fitted scores beat both hand-set rules.

One caveat on this comparison. The models are scored out-of-fold on unseen epitopes, which is fair, but the composite and the four-filter are the rules that built this library in the first place and were never fit to this assay. So the fair reading is that the learned weighting beats the hand-set one, and it is context-aware where the hand-set rule is not. The gap between logistic and gradient boosting is within the spread we saw in the held-out test (Table 5), so we do not read a winner between them.

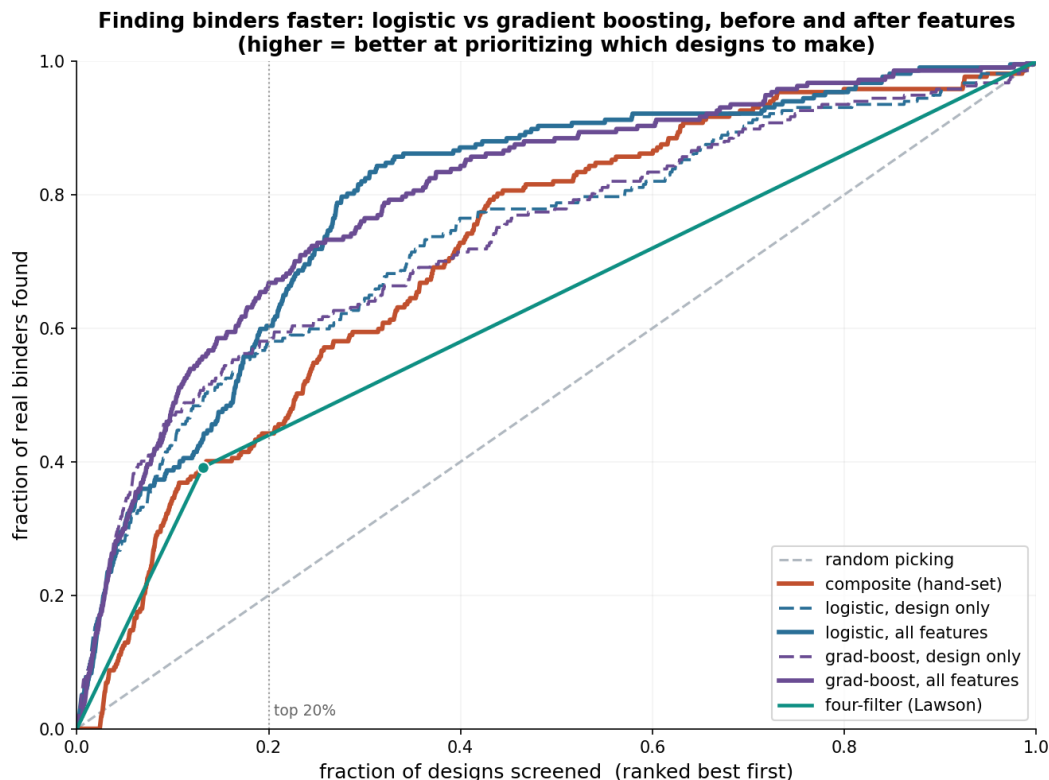


Figure 6: Finding binders faster than the composite. Rank all designs best-first by each score and track the fraction of real binders found as you screen deeper (x-axis); higher is better. Two model families (logistic in blue, gradient boosting in purple) are each shown with design metrics only (dashed) and the full feature set (solid), against the composite (orange) and random picking (diagonal). The four-filter (green) is a pass/fail set with no ranking inside it, so its honest gain is two straight segments through one elbow: make the 13% of designs that pass, catch 39% of the binders. At the top 20% (dotted line) the full-feature models catch the most, gradient boosting the most of all.

11 Summary and next step

The composite score was close to the right form already; it is a hand-set version of the model we can now fit. Refitting its weights on real binding barely helps on its own. The help comes from giving the model features of the epitope’s native structure: its secondary structure carries the largest single share of the signal, and among the dynamics features, rigidity in the bound state separates binders most. The best model is additive and interpretable: feature interactions add nothing that survives out-of-fold, so a per-feature scorecard captures the signal. Because the rigidity signal disappears when we remove the antibody and look at the epitope alone, we cannot yet separate “intrinsically pre-organized” from “clamped by the antibody” with static structures. Apo dynamics would settle it, and that is the same apo-versus-holo question the v3 campaign already exists to answer.

One caution applies throughout. 56 epitopes is a small sample for epitope-level features, every learned model shows a train-test gap, and graded affinity is close to unpredictable here. The binary in-or-out question is the one this readout can answer, and a simple context-aware model answers it better than the hand-set rule.

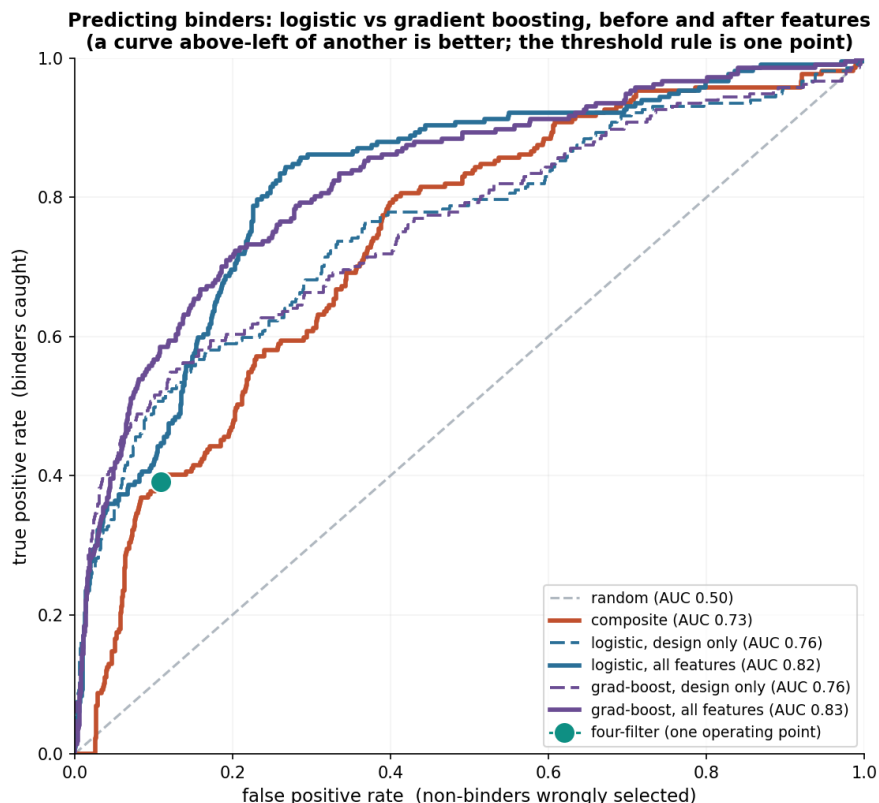


Figure 7: Predicting binders: model family and features, as ROC curves. Same binding truth for all. Each fitted score sweeps a curve (logistic blue, gradient boosting purple; dashed = design metrics only, solid = full feature set); the composite is orange and the four-filter is one point (false positive rate 0.11, true positive rate 0.39). Adding the native-context features lifts both families by about 0.06 AUC, and with the full set logistic (0.82) and gradient boosting (0.83) are within 0.01. Every fitted curve passes above and to the left of the composite (0.73) and the four-filter point, so the fitted scores beat both hand-set rules.

Reproducibility

Every number and figure here comes from a named script in `scripts/`, run on the joined assay data under `data/dp4_binding/`. Structural features need the `biotite` environment (`numpy<2`); the rest run under the repo's Python.

Script	Produces	Figure
<code>dp4_im0276_join.py</code>	assay-to-library join	(none)
<code>dp4_epitope_features.py</code>	native-context features	(none)
<code>dp4_composite_vs_binding.py</code>	composite vs binding	<code>composite_vs_binding.png</code>
<code>dp4_binder_ml_v2.py</code>	ablation $\times$ architecture	<code>binder_ml_v2.png</code>
<code>dp4_feature_shapley.py</code>	feature-block Shapley values	(none)
<code>dp4_interaction_test.py</code>	additive vs interactions, shapes	<code>additive_shapes.png</code>
<code>dp4_ebm.py</code>	EBM interaction check (needs <code>interpret</code>)	(none)
<code>dp4_headtohead.py</code>	held-out overfitting check	<code>headtohead.png</code>
<code>dp4_target_compare.py</code>	binary vs pseudo-affinity	<code>target_compare.png</code>
<code>dp4_context_rules.py</code>	context-dependent rules	<code>context_rules.png</code>
<code>dp4_model_vs_composite.py</code>	gain curve and ROC	<code>model_vs_composite.png</code> , <code>roc_comparison.png</code>